

COLLEGE FINAL YEAR CAPSTONE PROJECT DEFENSE

Smart Banking Management System

A C++17 Full-Stack Banking System Using OOP, DSA, REST API, React & MySQL

Student Name: [Student Name]
Roll No / USN: [Roll Number / USN]
Department: [Department of Computer Science & Engineering]
Institution: [College / University Name]
Academic Year: [Academic Year 2025 - 2026]

C++17 Engine

Winsock2 REST API

React 18 SPA

MySQL 3NF

Directed Graph + DFS

PBKDF2 Hashing

Problem Statement

Bridging the divide between theoretical textbook algorithms and real-world banking systems

Traditional Academic Limitations

- â€¢ **Trivial CLI Scopes:** Most educational banking projects focus strictly on primitive CLI balance mutations (add/subtract numbers).
- â€¢ **Monolithic Structures:** Zero separation of concerns; accounts, ledgers, and credentials are often merged into flat structs.
- â€¢ **Insecure Credentials:** Passwords and PINs are commonly stored as plaintext strings or rudimentary Caesar ciphers.
- â€¢ **Zero Concurrency & Networking:** No real-world web transport, network protocol handling, or REST APIs.
- â€¢ **Volatile or Flat Data:** Inability to demonstrate relational algebra, ACID guarantees, or 3NF normalization.

The Smart Banking Enterprise Paradigm

- â€¢ **Full-Stack Architecture:** Decoupled multi-tier system with C++17 core engine, Winsock2 REST API, and React 18 frontend.
- â€¢ **Advanced OOP & Polymorphism:** Abstract account hierarchies dynamically handling interest calculation and overdraft limits.
- â€¢ **Rigorously Benchmarked DSA:** 9 STL containers and graph algorithms solving real financial throughput problems.
- â€¢ **Cryptographic Security:** NIST SP 800-132 PBKDF2-HMAC-SHA256 salted PIN hashing with constant-time verification.
- â€¢ **Forensic AML Compliance:** Directed transaction graph modeling with automated cycle detection for wash trading.

Project Objectives

Concrete engineering deliverables achieved in the implementation

1. Modular C++17 Engine

Build a decoupled, thread-safe core engine using clean domain models, DTO contracts, and RAII memory safety.

2. OOP Principles

Demonstrate dynamic polymorphism, abstract base classes, encapsulation, inheritance, and standardized exception handling.

3. Real-World DSA

Implement and benchmark 9 STL structures and algorithms, delivering empirical proofs for theoretical Big-O complexities.

4. Cryptographic Auth

Implement salted PBKDF2-HMAC-SHA256 (10,000 rounds) and constant-time verification with zero plaintext credentials.

5. Native REST API

Engineer an embedded C++ Winsock2 HTTP server routing JSON requests and CORS negotiation without external web frameworks.

6. React 18 Interface

Deliver a modern, responsive Single Page Application with customer authentication, transfer forms, and live AML graphs.

7. MySQL 3NF Database

Formulate a Third Normal Form relational schema with cascade foreign keys, B-Tree indexes, and automated SQL dump sync.

8. Forensic AML Engine

Model inter-account fund flows via directed graphs and detect circular routing rings via Depth-First Search (DFS).

Technology Stack

Multi-tier modern fintech architecture built from first principles

C++17 Backend

- C++17 Standard
- STL Containers: vector, map, queue, stack, set
- Networking: Winsock2 Multi-threaded Sockets
- JSON DAL: Custom Native Serializer/Parser

React 18 Frontend

- React 18 SPA (Hooks)
- JavaScript (ES6+)
- HTML5 Canvas & SVG: Interactive Network Graph
- CSS3: Responsive Dark Fintech Theme

MySQL 3NF

- MySQL 8.0+ / MariaDB
- InnoDB Engine: Full ACID compliance
- 3NF Normalization: Zero transitive redundancy
- B-Tree Indexes: Sub-ms query acceleration

Dual Persistence

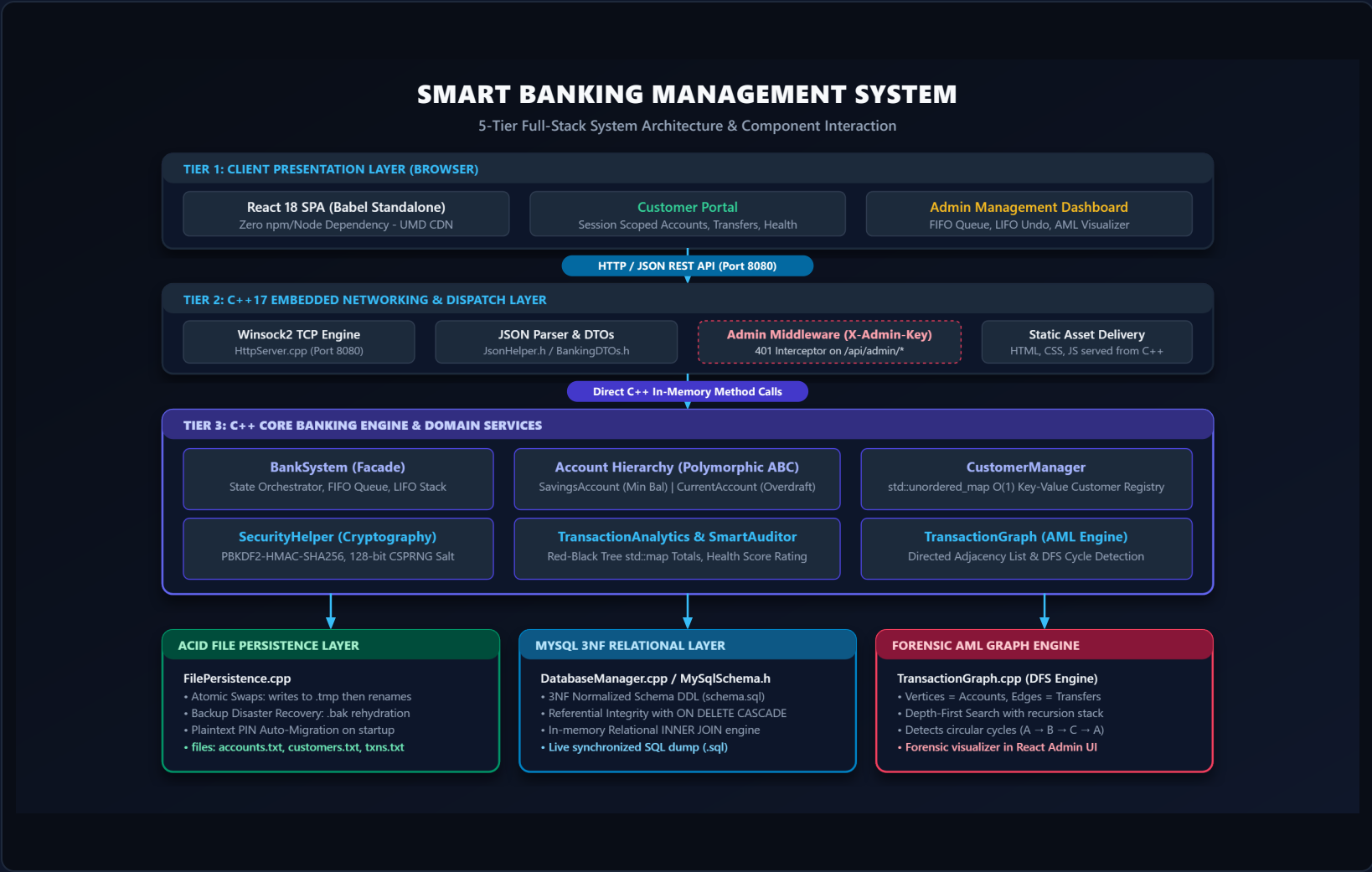
- Atomic Staging: .tmp write buffer
- Disaster Recovery: .bak mirror copies
- Atomic Swap: std::remove & std::rename
- SQL Sync: Live data dump serialization

Security & Auth

- PBKDF2-HMAC-SHA256: 10,000 rounds
- SPRNG Salt: 128-bit via CryptGenRandom
- Constant-Time: Bitwise XOR comparison
- Admin Middleware: X-Admin-Key filter

Main System Architecture

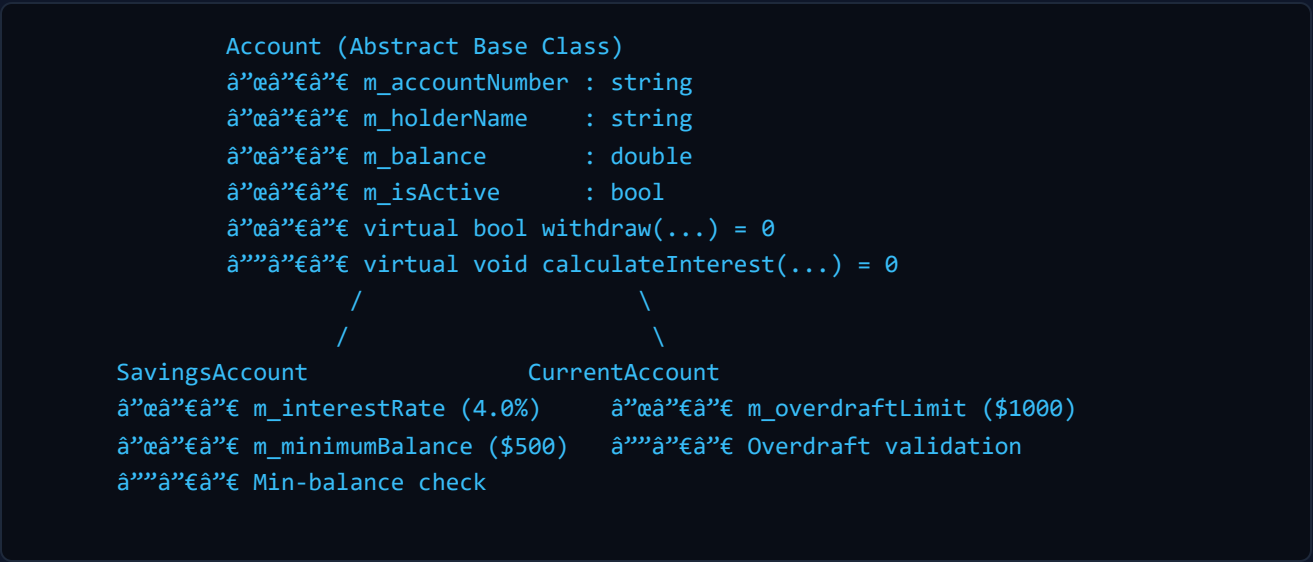
5-Tier End-to-End Enterprise Flow (docs/diagrams/01_system_architecture.svg)



Object-Oriented Programming (OOP) Design

Domain modeling, polymorphism, and behavioral specialization

Domain Class Hierarchy



Exception Hierarchy: BankingException → InvalidAmountException, InsufficientFundsException, AccountBlockedException, EntityNotFoundException.

Key OOP Principles Demonstrated

- **Encapsulation:** Private data members (`m_balance`, `m_pinHash`) are strictly protected. State mutations occur exclusively via validated public member functions.
- **Abstraction:** `Account` exposes a clean public interface (`deposit`, `withdraw`, `calculateInterest`) while encapsulating double-entry logging mechanics.
- **Inheritance:** `SavingsAccount` and `CurrentAccount` inherit common accounting logic while extending specialized banking fields.
- **Polymorphism & Virtual Dispatch:** Dynamic dispatch via `std::shared_ptr<Account>` ensures correct derived withdrawal limits and interest rates execute at runtime.
- **Centralized Exception Handling:** Domain exceptions are caught in `HttpServer.cpp` and mapped to HTTP status codes (400, 401, 403, 404, 422).

Data Structures & Algorithms Architecture

Container-to-problem mapping across the C++ engine (docs/diagrams/03_dsa_architecture.svg)

DATA STRUCTURES & ALGORITHMS (DSA) ARCHITECTURE

Mapping Real-World Banking Requirements to C++ STL Containers & Algorithmic Complexities

1. BANKING FUNCTIONAL NEED	2. C++ STL / CUSTOM DATA STRUCTURE	3. ALGORITHMIC COMPLEXITY
Transaction History Ledger Chronological account statement entries	<code>std::vector<Transaction></code> Contiguous buffer, maximal CPU cache locality	O(1) Amortized Append O(N) linear iteration for rendering
Customer & Account Lookup Instant retrieval by string ID	<code>std::unordered_map<string, shared_ptr<Account>></code> Hash table with bucket chaining (and CustomerManager)	O(1) Average Lookup Benchmarked ~430x speedup vs vector
Category Spending Aggregation Sorted financial expenditure summary	<code>std::map<string, double></code> Self-Balancing Red-Black Binary Search Tree	O(log N) Insert & Search In-order traversal yields alphabetical sort
Counterparty Deduplication Unique merchant / transaction ID sets	<code>std::set<string></code> Balanced Binary Search Tree with unique keys	O(log N) Insertion Duplicates automatically rejected
Pending Request Approval Pipeline Loan applications & unblock requests	<code>std::queue<BankingRequest></code> First-In, First-Out (FIFO) pipeline	O(1) Enqueue / Dequeue Guarantees strict submission fairness
Transaction Reversal & Undo Multi-step cascading rollback engine	<code>std::stack<TransactionReversal></code> Last-In, First-Out (LIFO) compensation stack	O(1) Push / Pop Reverses operations in reverse order
Chronological Date-Range Search Filtering ledgers by timestamp	<code>std::lower_bound</code> (Binary Search) Logarithmic boundary search over sorted vectors	O(log N) Comparisons Benchmarked ~265x-375x speedup
Forensic AML Transaction Graph • Inter-account capital flow modeling • Forensic Anti-Money Laundering analysis • Circular fund transfer ring detection Wash-trading & layering patterns	Directed Graph + DFS Cycle Detection 1. Graph: <code>unordered_map<string, vector<Edge>></code> Vertices V = Accounts, Directed Edges E = Transfers 2. Traversal: Depth-First Search (DFS) Three-color recursion-stack state tracking	O(V + E) Linear Time • Space: O(V + E) Adjacency List • Cycle Finder: O(V + E) DFS • Output: A → B → C → A routing rings • Visualized live in React Dashboard

DSA Complexity & Algorithmic Rigor

Theoretical bounds and measured performance benchmarks

Data Structure / Algorithm	Actual Banking Application	Time Complexity	Space Complexity
<code>std::vector<Transaction></code>	Chronological transaction history ledger	O(1) amortized append	O(N) contiguous
<code>std::unordered_map<string, Account*></code>	Direct customer and account lookup	O(1) average (O(N) worst)	O(N) hash buckets
<code>std::map<string, double></code>	Sorted category spending summary	O(log K) insert/search	O(K) Red-Black tree
<code>std::set<string></code>	Unique counterparty deduplication	O(log U) uniqueness	O(U) balanced BST
<code>std::queue<BankingRequest></code>	Administrative request approval pipeline	O(1) enqueue/dequeue	O(R) sequential
<code>std::stack<TransactionReversal></code>	Cascading transaction undo engine	O(1) push/pop	O(S) sequential
Binary Search (<code>std::lower_bound</code>)	Date-range queries on sorted ledgers	O(log N) comparisons	O(1) auxiliary
Directed Graph (Adjacency List)	Transaction capital flow network	O(1) add edge	O(V + E) adj list
DFS Cycle Detection	Forensic AML circular ring detection	O(V + E) traversal	O(V) recursion

Important Academic Distinction: Measured benchmark performance (e.g. Binary Search executing ~293x faster than linear scan across 5,000 items) and asymptotic Big-O complexity (O(log N) vs O(N)) are fundamentally different concepts. Benchmarks measure hardware execution speed on specific CPUs; Big-O defines mathematical growth limits.

Core Banking Operations & Workflow

Transactional integrity, balance enforcement, and cascading rollbacks



Account Specialization

- â€” **Savings Account:** Enforces minimum balance (\$500.00). Withdrawals violating threshold are rejected with HTTP 422.
- â€” **Current Account:** Authorized overdraft limit (\$1,000.00). Permits negative balances up to authorized threshold.
- â€” **Account State:** Blocked accounts reject all transactions with HTTP 403 Forbidden.

Atomic Fund Transfers

- â€” **Validation:** Verifies source/destination accounts and checks self-transfers (returns HTTP 400).
- â€” **Synchronous Execution:** Debits source (TRANSFER_OUT) and credits destination (TRANSFER_IN) atomically.
- â€” **Audit Logging:** Appends dual entries with unique transaction IDs to immutable ledgers.

LIFO Reversal Stack

- â€” **Non-Destructive Undo:** Banking compliance forbids deleting history; reversals execute compensating financial entries.
- â€” **LIFO Sequence:** Stack pops the most recent operation first, ensuring intermediate dependencies remain valid.
- â€” **Balance Restoration:** Perfectly restores pre-transaction account balances.

Smart Analytics & Anti-Money Laundering (AML)

Financial health grading and forensic graph cycle detection

Financial Health Auditor (SmartAuditor)

â€¢ **Inflow & Outflow Tracking:** Aggregates deposits, salary credits, withdrawals, and merchant payments.

â€¢ **Savings Rate Formulation:**

$$\text{Savings Rate} = (\text{Net Savings} / \text{Total Inflow}) * 100\%$$

â€¢ **Health Grading:** Grades account health: EXCELLENT, HEALTHY, MODERATE, NEEDS_ATTENTION, CRITICAL.

â€¢ **Automated Advisory:** Delivers actionable financial guidance (e.g. building 6-month emergency liquidity reserves).

AML Transaction Graph & Cycle Detection

â€¢ **Graph Modeling:** Accounts form Vertices (V); inter-account fund transfers form Directed Edges ($E = u \rightarrow v$) with monetary weights.

â€¢ **Wash Trading & Layering:** Criminals route dirty money through multiple intermediary accounts to obscure origin.

â€¢ **Color DFS Cycle Algorithm:**

$$\text{Account A} \rightarrow \text{Account B} \rightarrow \text{Account C} \rightarrow \text{Account A}$$

Uses recursion-stack state tracking (visited and inStack) in $O(V + E)$ time.

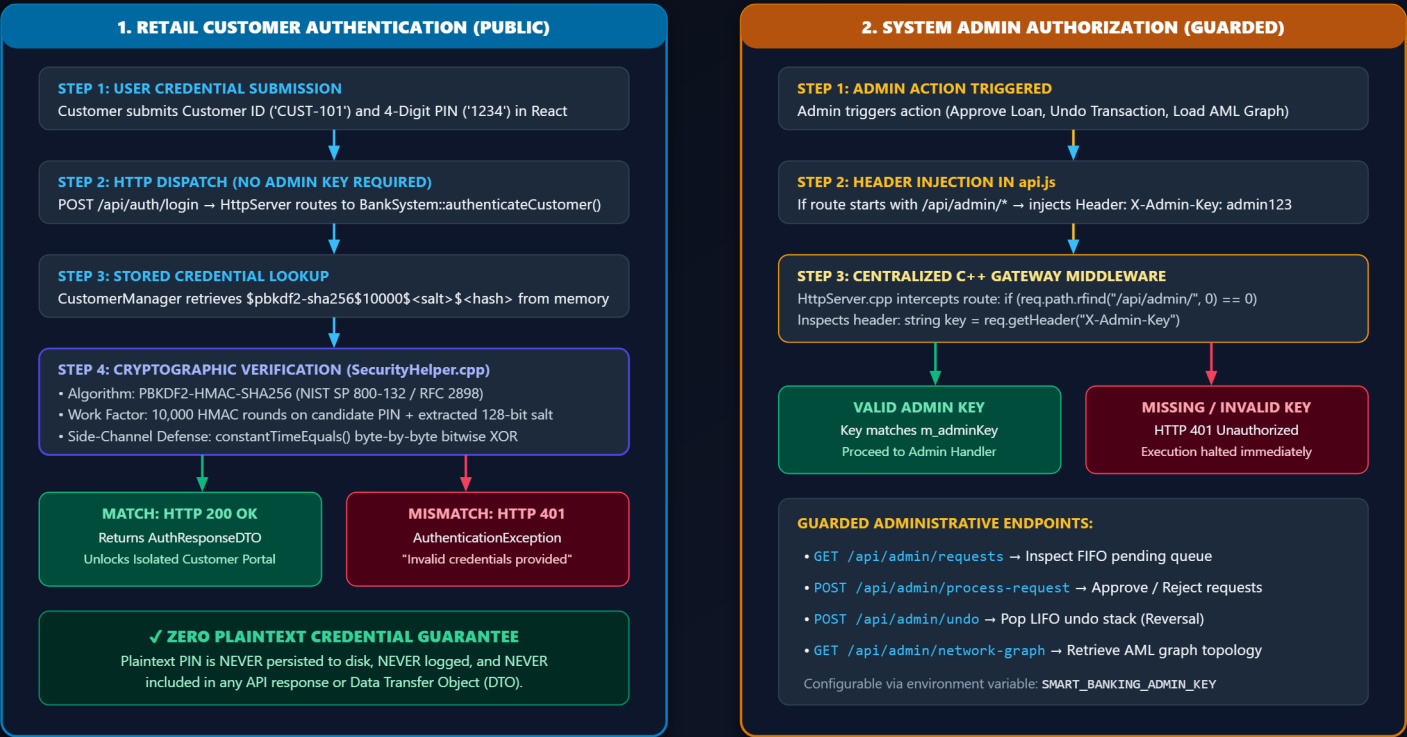
â€¢ **Live React Visualizer:** Renders the directed network dynamically and flags laundering rings with red alert badges.

Authentication & Security Architecture

Cryptographic PIN hashing vs Admin API authorization separation (docs/diagrams/02_auth_security_flow.svg)

AUTHENTICATION & AUTHORIZATION ARCHITECTURE

Complete Separation of Customer PBKDF2 PIN Verification & Admin X-Admin-Key Authorization



Customer PBKDF2 PIN Security

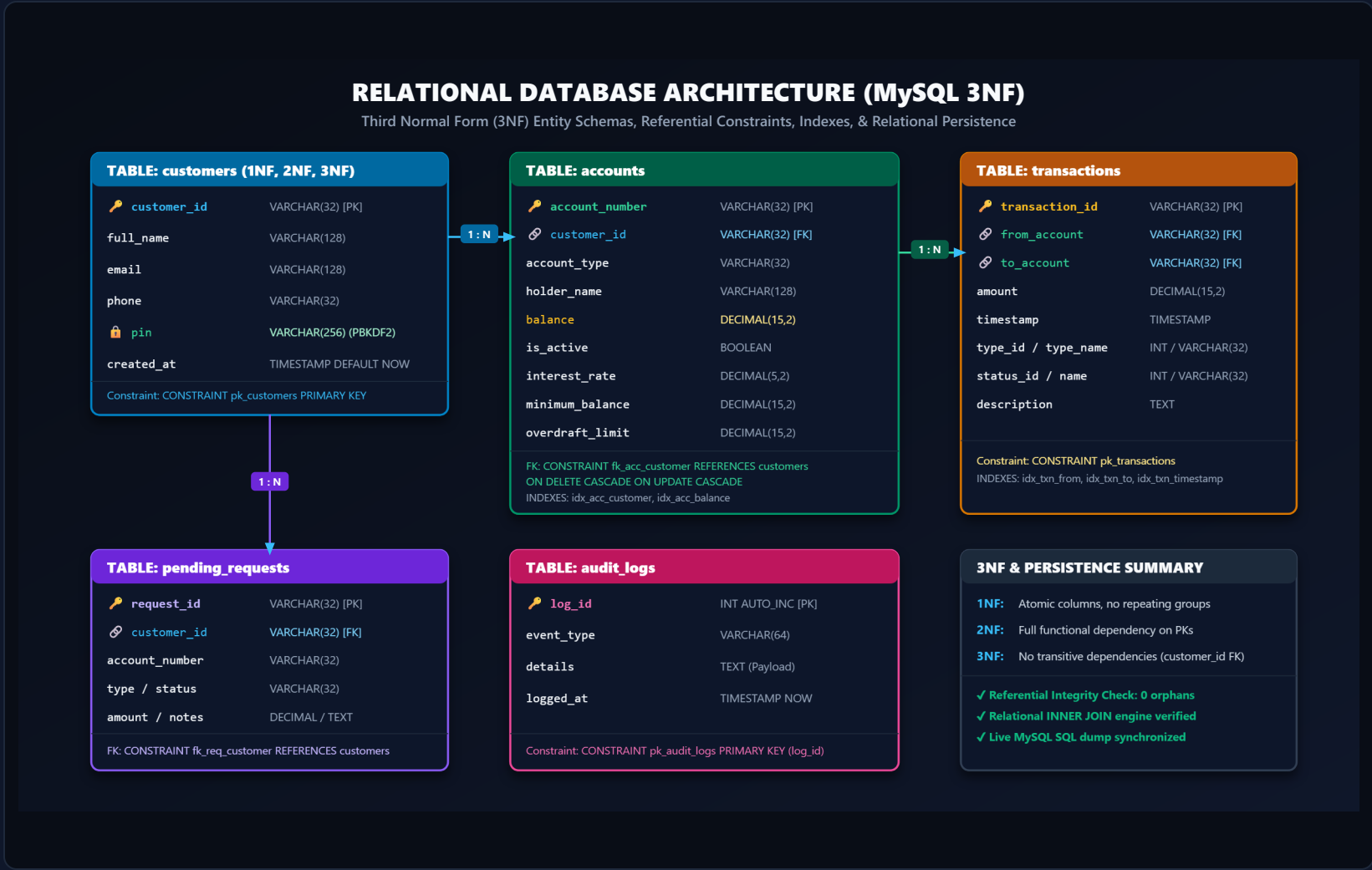
- **Key Stretching:** 10,000 rounds of HMAC-SHA256 (NIST SP 800-132).
- **Unique Salt:** 128-bit CSPRNG salt generated via Windows CryptoAPI CryptGenRandom.
- **Constant-Time Equals:** Bitwise XOR byte comparison neutralizes side-channel timing attacks.
- **Zero Plaintext Exposure:** Stored as \$pbkdf2-sha256\$10000\$<salt>\$<hash>. Never exposed in API payloads or SQL dumps.

Admin API Authorization

- **Centralized Middleware:** Intercepts all /api/admin/* routes.
- **Header Inspection:** Validates incoming X-Admin-Key header.
- **Strict Rejection:** Invalid keys immediately return HTTP 401 Unauthorized.
- **Customer Decoupling:** Customer endpoints never require or expose admin credentials.

Relational Database Architecture (MySQL 3NF)

Entity normalization, foreign key constraints, and B-Tree indexing (docs/diagrams/04_database_architecture.svg)



Testing & Verification Results

Rigorous automated verification suite: 12 / 12 Test Suites PASSED (100% Integrity)

[PASS] ALL 12 VERIFICATION TEST SUITES PASSED WITH 100% INTEGRITY

0 Warnings | 0 Errors | GCC 6.3.0 C++17

OOP & Authentication

- â€¢ [1/12] OOP Polymorphic Dispatch: PASSED
- â€¢ [2/12] Customer PBKDF2 Auth: PASSED
- â€¢ [3/12] std::queue FIFO Order: PASSED
- â€¢ [4/12] std::stack LIFO Rollback: PASSED

DSA & AML Algorithms

- â€¢ [5/12] std::map Category Summary: PASSED
- â€¢ [6/12] std::set Deduplication: PASSED
- â€¢ [7/12] Directed Graph & DFS Cycles: PASSED
- â€¢ [8/12] Binary Search $O(\log N)$: PASSED

Full-Stack & Persistence

- â€¢ [9/12] C++ REST API Routing: PASSED
- â€¢ [10/12] MySQL 3NF Schema & Dump: PASSED
- â€¢ [11/12] Frontend Static Asset Delivery: PASSED
- â€¢ [12/12] End-to-End ACID Workflow: PASSED

Limitations & Future Scope

Distinguishing the educational prototype scope from production enterprise infrastructure

Current Educational Limitations

- â€¢ **Transport Security:** The embedded Winsock2 server operates over HTTP. Production banking mandates TLS 1.3 / HTTPS to encrypt network traffic.
- â€¢ **Memory-Hard KDF:** PBKDF2 was chosen for portability; enterprise deployments prefer **Argon2id** (PHC winner) to resist ASIC/FPGA hardware attacks.
- â€¢ **Administrative Auth:** Uses shared X-Admin-Key header; production requires asymmetric RS256 JWTs or OAuth2 bearer tokens with refresh rotation.
- â€¢ **Rate Limiting:** No exponential backoff or 3-strike account lockout against brute-force attacks.
- â€¢ **Persistence:** Flat-file staging is designed for standalone offline operation, not multi-node clustered persistence.

Future Engineering Enhancements

- â€¢ **HTTPS / TLS 1.3 Sockets:** Integrate OpenSSL / Windows SChannel to encrypt data in transit.
- â€¢ **Argon2id Password Hashing:** Upgrade key derivation with memory-hard parameters (\$m=64\text{MB}\$, \$t=3\$, \$p=4\$).
- â€¢ **Account Lockout Policy:** Enforce 3-strike lockout and CAPTCHA challenges on consecutive failed PIN attempts.
- â€¢ **Machine Learning AML Scoring:** Real-time Graph Neural Network (GNN) transaction scoring for fraud classification.
- â€¢ **Cloud Microservices:** Decompose services with Docker containers and deploy on Kubernetes.

PROJECT CONCLUSION & VIVA DEFENSE

Smart Banking Management System

Successfully demonstrates how modern C++17 systems programming, classical Object-Oriented principles, optimal Data Structures & Algorithms, cryptographic security, and 3NF database normalization can seamlessly power a modern, responsive web application.

Thank You!

Questions & Discussion

Docs: [docs/dsa_documentation.md](#)

Docs: [docs/database_documentation.md](#)

Diagrams: [docs/diagrams/](#)

Test Suite: `SmartBankingSystem.exe --test`